

LLAMA3

lm3.py（下载，安装，部署，应用）

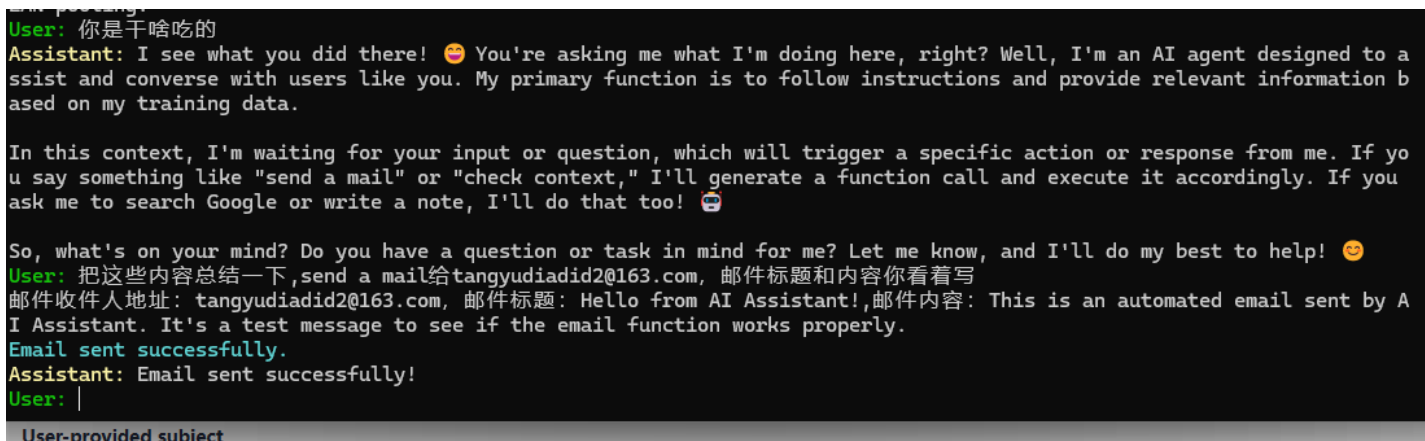
1.<https://ollama.com/>

下载，安装，ollama run llama3:8b 默认端口 <http://localhost:11434/v1>

2.SentenceTransformer 加载 all-MiniLM-L6-v2 ，手动下载：<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/main>

3.function-calling: 这个可以参考openai的写法，
<https://platform.openai.com/docs/guides/function-calling>

4.测试技能



```
User: 你是干啥吃的
Assistant: I see what you did there! 😊 You're asking me what I'm doing here, right? Well, I'm an AI agent designed to assist and converse with users like you. My primary function is to follow instructions and provide relevant information based on my training data.

In this context, I'm waiting for your input or question, which will trigger a specific action or response from me. If you say something like "send a mail" or "check context," I'll generate a function call and execute it accordingly. If you ask me to search Google or write a note, I'll do that too! 📧

So, what's on your mind? Do you have a question or task in mind for me? Let me know, and I'll do my best to help! 😊
User: 把这些内容总结一下,send a mail给tangyudiadid2@163.com, 邮件标题和内容你看着写
邮件收件人地址: tangyudiadid2@163.com, 邮件标题: Hello from AI Assistant!,邮件内容: This is an automated email sent by AI Assistant. It's a test message to see if the email function works properly.
Email sent successfully.
Assistant: Email sent successfully!
User: |
User-provided subject
```

app.py(RAG)

1.pip install -r require.txt

2.安装docker (为了装向量数据库) <https://docs.docker.com/desktop/install/windows-install/>

docker pull phidata/pgvector:16

docker volume create pgvolume

docker run -itd -e POSTGRES_DB=ai -e POSTGRES_USER=ai -e POSTGRES_PASSWORD=ai -e PGDATA=/var/lib/postgresql/data/pgdata -v pgvolume:/var/lib/postgresql/data -p 5532:5432 --name pgvector phidata/pgvector:16

#3.GROQ_API_KEY <https://console.groq.com/keys> 配置好自己的key 这样就可以用70B的了

4.ollama run nomic-embed-text 知识库向量化，这个感觉一般，也可以换openai的

5.streamlit run app.py

LLAMA3微调-量化-部署-应用 一条龙

微调

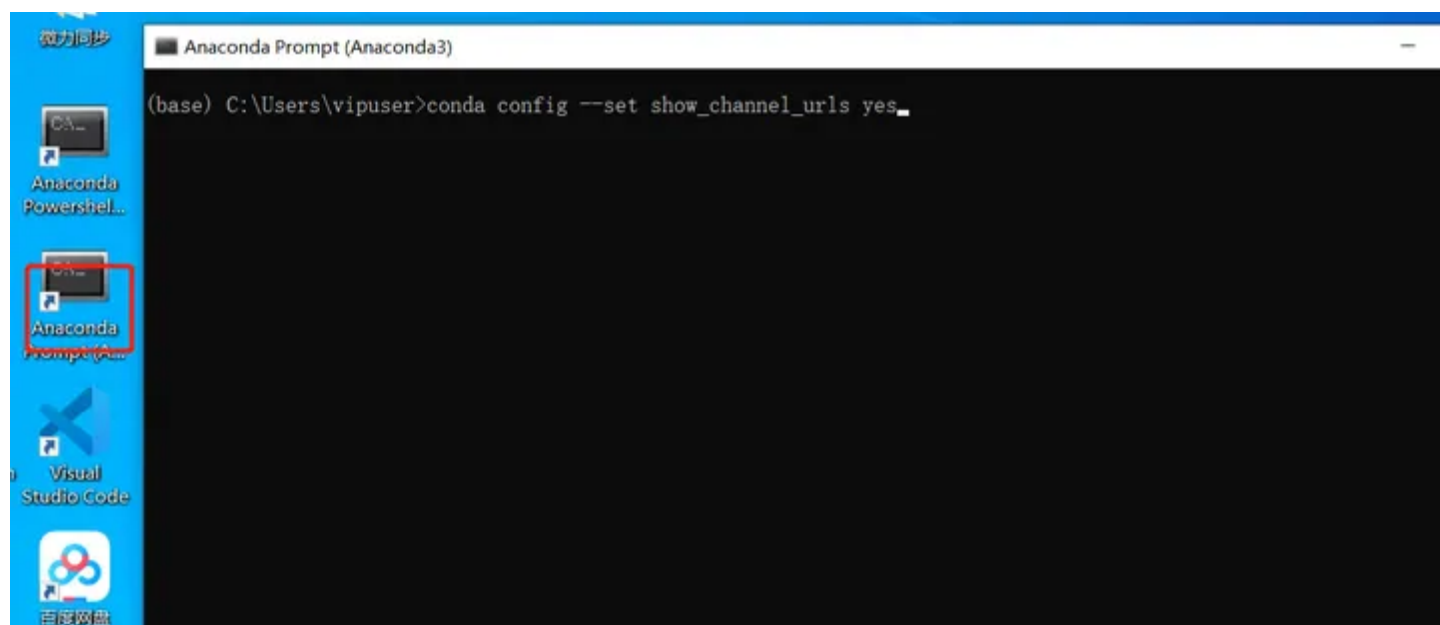
1.先下载一个完整版本，官网版或者别人微调过的中文版也可以的，例如：

<https://huggingface.co/hfl/llama-3-chinese-8b-instruct-v2/tree/main>

下载所有files文件，保存到一个文件夹里面，这个可以任选模型，huggingface上好多的

新建环境：

conda config --set show_channel_urls yes



然后执行dir命令，可以看到文件名.condarc 文件

```
Anaconda Prompt (Anaconda3)
(base) C:\Users\vipuser>dir
驱动器 C 中的卷没有标签。
卷的序列号是 E8FD-A2C1

C:\Users\vipuser 的目录
2022/03/29  14:33    <DIR>          .
2022/03/29  14:33    <DIR>          ..
2022/01/04  17:16                28 .bash_history
2022/01/03  13:05    <DIR>          .conda
2022/03/29  14:33    25 .condarc
2022/01/03  16:04    <DIR>          .keras
2022/01/10  12:14       207 .python_history
2022/01/03  00:24    <DIR>          .vscode
2021/12/30  11:54    <DIR>          3D Objects
2021/12/30  11:54    <DIR>          Contacts
2022/01/10  13:32    <DIR>          Desktop
2022/01/04  15:16    <DIR>          Documents
2022/01/04  17:19    <DIR>          Downloads
2021/12/30  11:54    <DIR>          Favorites
2021/12/30  11:54    <DIR>          Links
2021/12/30  11:54    <DIR>          Music
2021/12/30  11:54    <DIR>          Pictures
2021/12/30  11:54    <DIR>          Saved Games
2021/12/30  11:55    <DIR>          Searches
2021/12/30  11:54    <DIR>          Videos
                3 个文件          260 字节
                17 个目录 55,325,364,224 可用字节

(base) C:\Users\vipuser>
```

在C:\Users\vipuser目录下编辑更改为以下源，用记事本打开，修改保存。

default_channels:

- <https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main>
- <https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free>
- <https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/r>

conda create -n llama3 python==3.10.11

pip install -r requirements.txt -i <https://pypi.tuna.tsinghua.edu.cn/simple> --no-dependencies

2.指令微调，执行run_clm_sft_with_peft，所需参数：

代码块

```
1  --model_name_or_path
2  D:\PycharmProject\2024\llama-3-chinese-8b-instruct-v2
3  --tokenizer_name_or_path
4  D:\PycharmProject\2024\llama-3-chinese-8b-instruct-v2
```

```
5  --dataset_dir
6  D:\\PycharmProject\\2024\\Chinese-LLaMA-Alpaca-3-main\\data
7  --per_device_train_batch_size
8  1
9  --per_device_eval_batch_size
10 1
11 --do_train
12 1
13 --do_eval
14 1
15 --seed
16 42
17 --bf16
18 1
19 --num_train_epochs
20 3
21 --lr_scheduler_type
22 cosine
23 --learning_rate
24 1e-4
25 --warmup_ratio
26 0.05
27 --weight_decay
28 0.1
29 --logging_strategy
30 steps
31 --logging_steps
32 10
33 --save_strategy
34 steps
35 --save_total_limit
36 3
37 --evaluation_strategy
38 steps
39 --eval_steps
40 100
41 --save_steps
42 200
43 --gradient_accumulation_steps
44 8
45 --preprocessing_num_workers
46 8
47 --max_seq_length
48 1024
49 --output_dir
50 D:\\PycharmProject\\2024\\llama3-lora
51 --overwrite_output_dir
```

```

52 1
53 --ddp_timeout
54 30000
55 --logging_first_step
56 True
57 --lora_rank
58 64
59 --lora_alpha
60 128
61 --trainable
62 "q_proj,v_proj,k_proj,o_proj,gate_proj,down_proj,up_proj"
63 --lora_dropout
64 0.05
65 --modules_to_save
66 "embed_tokens,lm_head"
67 --torch_dtype
68 bfloat16
69 --validation_file
70 D:\\PycharmProject\\2024\\Chinese-LLaMA-Alpaca-3-
    main\\eval\\ruozhiba_qa2449_gpt4turbo.json
71 --load_in_kbits
72 16
73

```

如果显存还是不够，可以改这俩

--load_in_kbits

8

--modules_to_save

"None"

3.合并LORA，现在只训练和保存了一部分权重，需要和原始的合并在一起

1.执行merge_llama3_with_chinese_lora_low_mem.py，需要传入的参数（改成自己的）：

--base_model

D:\\PycharmProject\\2024\\llama-3-chinese-8b-instruct-v2

--lora_model

D:\\PycharmProject\\2024\\llama3-lora

--output_dir

D:\\PycharmProject\\2024\\llama3-lora-merge

量化

1.用llama.cpp (<https://github.com/ggerganov/llama.cpp>) 这个项目

需要先装好CMAKE: <https://cmake.org/download/>

然后

```
git clone https://github.com/ggerganov/llama.cpp
```

```
cd llama.cpp
```

```
pip install -r requirements/requirements-convert-hf-to-gguf.txt
```

```
cmake -B build
```

```
cmake --build build --config Release
```

2.在项目文件里面找到咱们要用的转换工具，

```
convert-hf-to-gguf.py D:\\PycharmProject\\2024\\llama3-lora-merge --outtype f16 --outfile  
D:\\PycharmProject\\2024\\my_llama3.gguf
```

3.进入到这个路径D:\\PycharmProject\\2024\\test_llama3.cpp\\llama.cpp\\build\\bin\\Release

```
quantize.exe D:\\PycharmProject\\2024\\my_llama3.gguf
```

```
D:\\PycharmProject\\2024\\quantized_model.gguf q4_0
```

部署

ollama,lmstudio都是比较简单的方法，用法都差不多，而且都自带启动服务

```
ollama create 名字 -f Modelfile
```

量化colab

1.!git clone <https://github.com/ggerganov/llama.cpp>

先下载这个项目，GG哥们太狠了

```
2.!cd llama.cpp && LLAMA_CUBLAS=1 make && pip install -r requirements/requirements-  
convert-hf-to-gguf.txt
```

编译安装

3.下载huggingface上的模型，注意coloab内存有限制，如果在这上面只能搞小的

```
from huggingface_hub import snapshot_download

model_name = "Qwen/Qwen1.5-1.8B"

methods = ['q4_k_m']

base_model = "./original_model2/"

quantized_path = "./quantized_model/"

snapshot_download(repo_id=model_name, local_dir=base_model ,
local_dir_use_symlinks=False)

original_model = quantized_path+'FP16.gguf'
```

4.执行量化

```
!mkdir ./quantized_model/

!python llama.cpp/convert-hf-to-gguf.py ./original_model2/ --outtype f16 --outfile
./quantized_model/FP16.gguf
```

5.模型可以上传到huggingface,方便以后下载

```
from huggingface_hub import notebook_login

notebook_login()

from huggingface_hub import HfApi, HfFolder, create_repo, upload_file

model_path = "./quantized_model/Q4_K_M.gguf" # Your model's local path

repo_name = "qwen1.5-llm" # Desired HF Hub repository name

repo_url = create_repo(repo_name, private=False)

api = HfApi()

api.upload_file(

    path_or_fileobj=model_path,

    path_in_repo="Q4_K_M.gguf",

    repo_id="skuma307/qwen1.5-llm",

    repo_type="model",

)
```

